

Problem Statement:

This dataset includes details of applicants who have applied for loan. The dataset includes details like credit history, loan amount, their income, dependents etc.

Independent Variables:

- Loan_ID
- Gender
- Married
- Dependents
- Education
- Self_Employed
- ApplicantIncome
- CoapplicantIncome
- Loan_Amount
- Loan_Amount_Term
- Credit History
- Property_Area

Dependent Variable (Target Variable):

- Loan_Status

You have to build a model that can predict whether the loan of the applicant will be approved or not on the basis of the details provided in the dataset.

- Importing require library for performing EDA, Data Wrangling and data cleaning

```
In [1]: import pandas as pd # for data wrangling purpose
import numpy as np # Basic computation library
import seaborn as sns # For Visualization
import matplotlib.pyplot as plt # plotting package
import warnings # Filtering warnings
warnings.filterwarnings('ignore')
```

C:\Users\Prashant\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
from pandas.core import (

```
In [2]: # Importing Loan Predication CSV dataset file using pandas
df=pd.read_csv('loan_prediction.csv')
```

```
In [3]: # print('No of Rows:',df.shape[0])
# print('No. of Columns:',df.shape[1])
df.head()
```

```
Out[3]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0

```
In [4]: df.tail()
```

```
Out[4]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	36
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	18
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	36
612	LP002984	Male	Yes	2	Graduate	No	7583	0.0	187.0	36
613	LP002990	Female	No	0	Graduate	Yes	4583	0.0	133.0	36

```
In [5]: df.columns
```

```
Out[5]: Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',  
          'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',  
          'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],  
          dtype='object')
```

Statistical Analysis

Before Going for Statistical exploration of data, first check integrity of data & Missing value

Data Integrity Check

```
In [6]: df.duplicated().sum() # This check any if any duplicated entry exit in dataset
```

```
Out[6]: 0
```

Since dataset is large, Let check for any entry which is repeated or duplicated in dataset at same date.

Comment:

Dataset doesnot contain Any duplicate entry. So Yes To Go !!!

Datatype Check

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Loan_ID               614 non-null    object
 1   Gender                601 non-null    object
 2   Married               611 non-null    object
 3   Dependents            599 non-null    object
 4   Education             614 non-null    object
 5   Self_Employed         582 non-null    object
 6   ApplicantIncome       614 non-null    int64
 7   CoapplicantIncome     614 non-null    float64
 8   LoanAmount            592 non-null    float64
 9   Loan_Amount_Term      600 non-null    float64
10   Credit_History         564 non-null    float64
11   Property_Area         614 non-null    object
12   Loan_Status           614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

- **Comment :**

- In loan application status dataset we have 614 rows with 13 columns including target variable.
- A Target Variable is 'Loan_Status' having object datatype and It is categorical variable.
- Gender, Married, Education, Self Employed, Credit History, Loan Status are categorical features.
- There are three types of datatype dtypes: float64(4), int64(1), object(8)

Missing value check

```
In [8]: # missing_values = df.isnull().sum().sort_values(ascending = False)
# percentage_missing_values =(missing_values/len(df))*100
# print(pd.concat([missing_values, percentage_missing_values], axis =1, keys =['Missing Values', '% Missing data']))
df.isnull().sum()
```

```
Out[8]: Loan_ID      0
Gender      13
Married     3
Dependents  15
Education   0
Self_Employed 32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount  22
Loan_Amount_Term 14
Credit_History 50
Property_Area 0
Loan_Status 0
dtype: int64
```

- **Comment :**

- 7 out 13 columns contains missing value.
- As small amount of data is missing so we use mean and mode to replace with NaN values.

Lets explore categorical features before missing value imputation.

Start with Enlisting Value counts & Sub-categories of different categorical features available

```
In [9]: # category=['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
#               'Loan_Amount_Term', 'Property_Area', 'Credit_History', 'Loan_Status']

df.drop("Loan_ID", axis=1, inplace=True)
category=df.select_dtypes(include="object")

for i in category:
    print(i)
    print(df[i].value_counts())
    print('='*100)
```

Gender
Gender
Male 489
Female 112
Name: count, dtype: int64

Married
Married
Yes 398
No 213
Name: count, dtype: int64

Dependents
Dependents
0 345
1 102
2 101
3+ 51
Name: count, dtype: int64

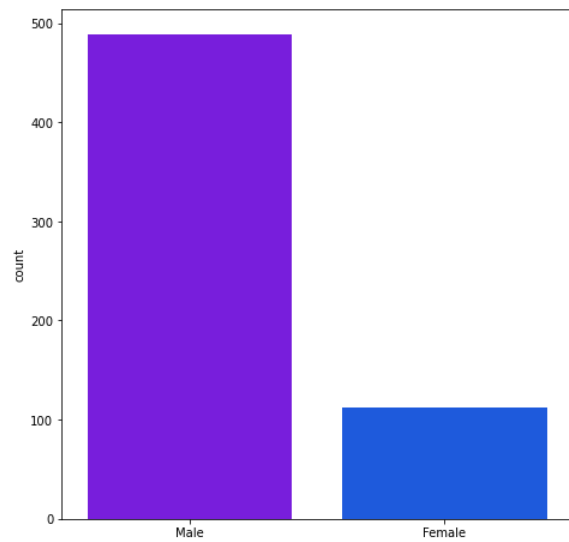
Education
Education
Graduate 480
Not Graduate 134
Name: count, dtype: int64

Self_Employed
Self_Employed
No 500
Yes 82
Name: count, dtype: int64

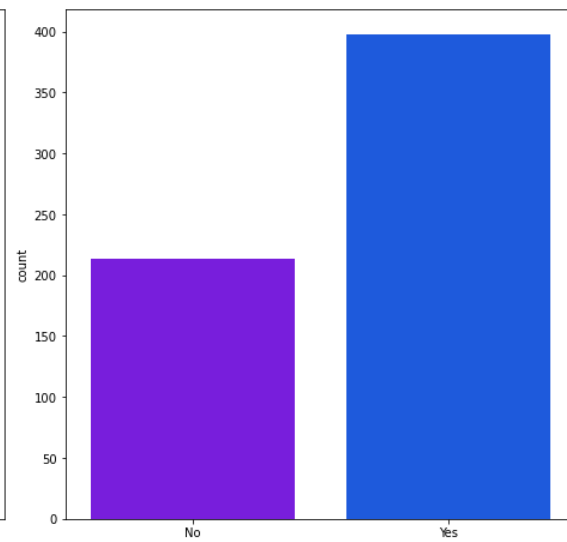
Property_Area
Property_Area
Semiurban 233
Urban 202
Rural 179
Name: count, dtype: int64

Loan_Status
Loan_Status
Y 422
N 192
Name: count, dtype: int64

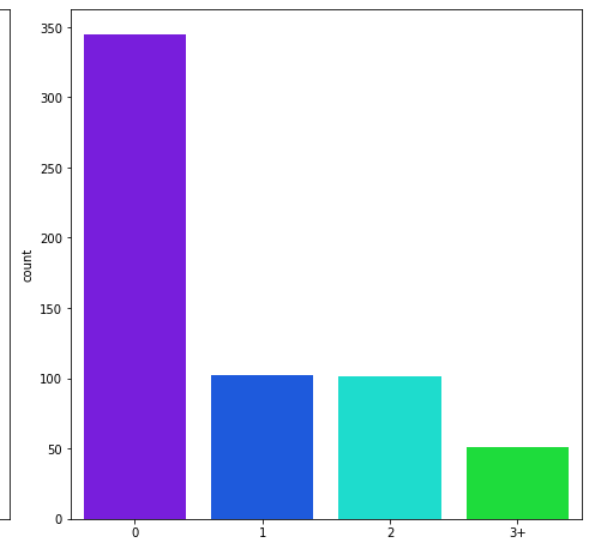
```
In [17]: sns.set_palette('gist_rainbow_r')
plt.figure(figsize=(20,20),facecolor='white')
plotnumber=1
# category=['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
#           'Loan_Amount_Term', 'Property_Area', 'Credit_History', 'Loan_Status']
for i in category:
    if plotnumber<=7:
        ax=plt.subplot(3,3,plotnumber)
        sns.countplot(x=df[i])
        plt.xlabel(i,fontsize=20)
        plotnumber+=1
plt.tight_layout()
plt.show()
```



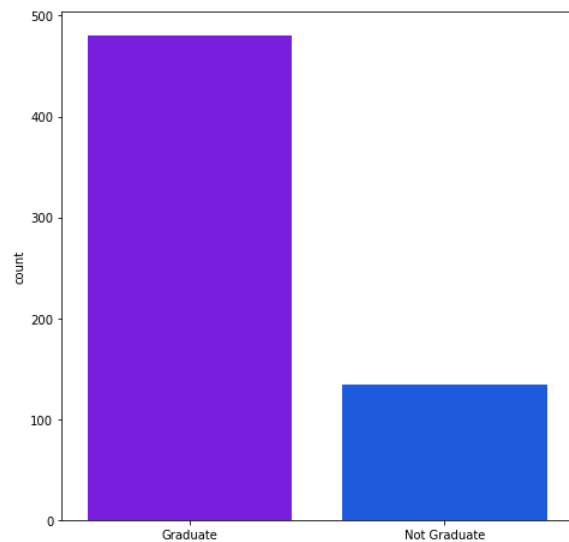
Gender



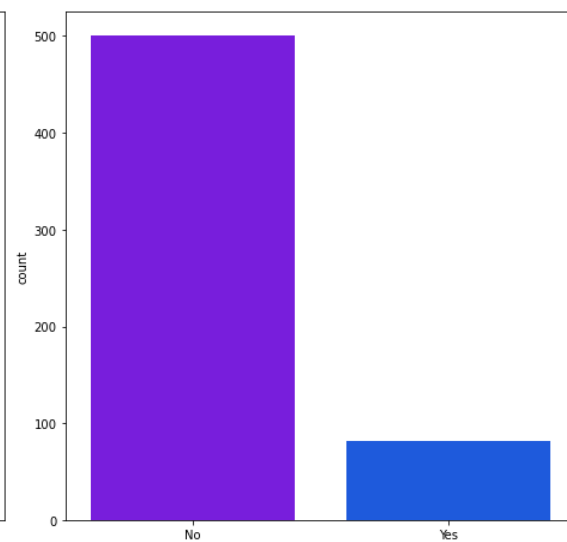
Married



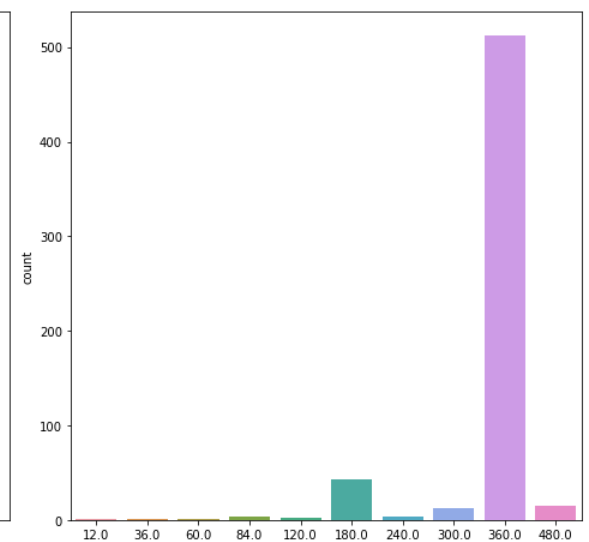
Dependents



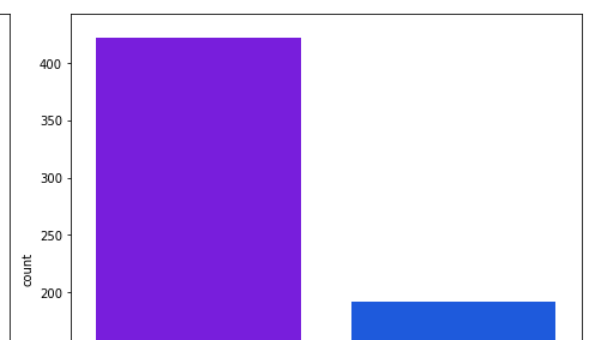
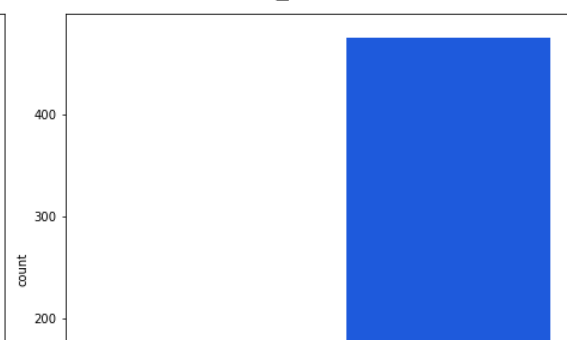
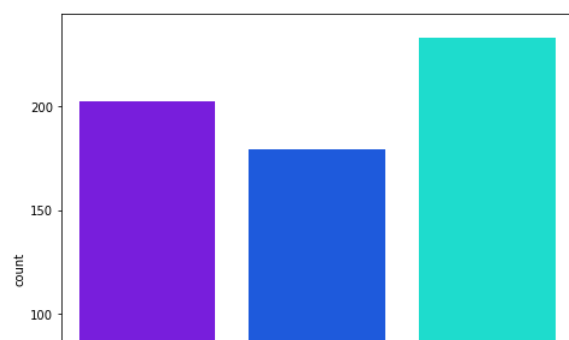
Education

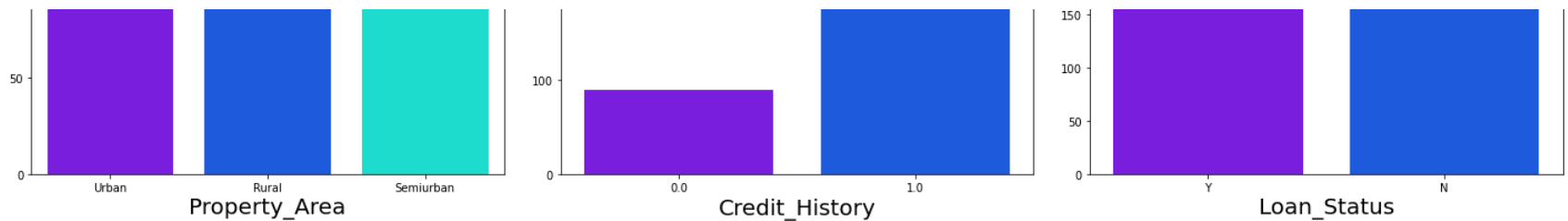


Self_Employed



Loan_Amount_Term





```
In [ ]: df["Loan_Amount_Term"].unique()
```

```
In [ ]: df["Credit_History"].unique()
```

- **Comment:**

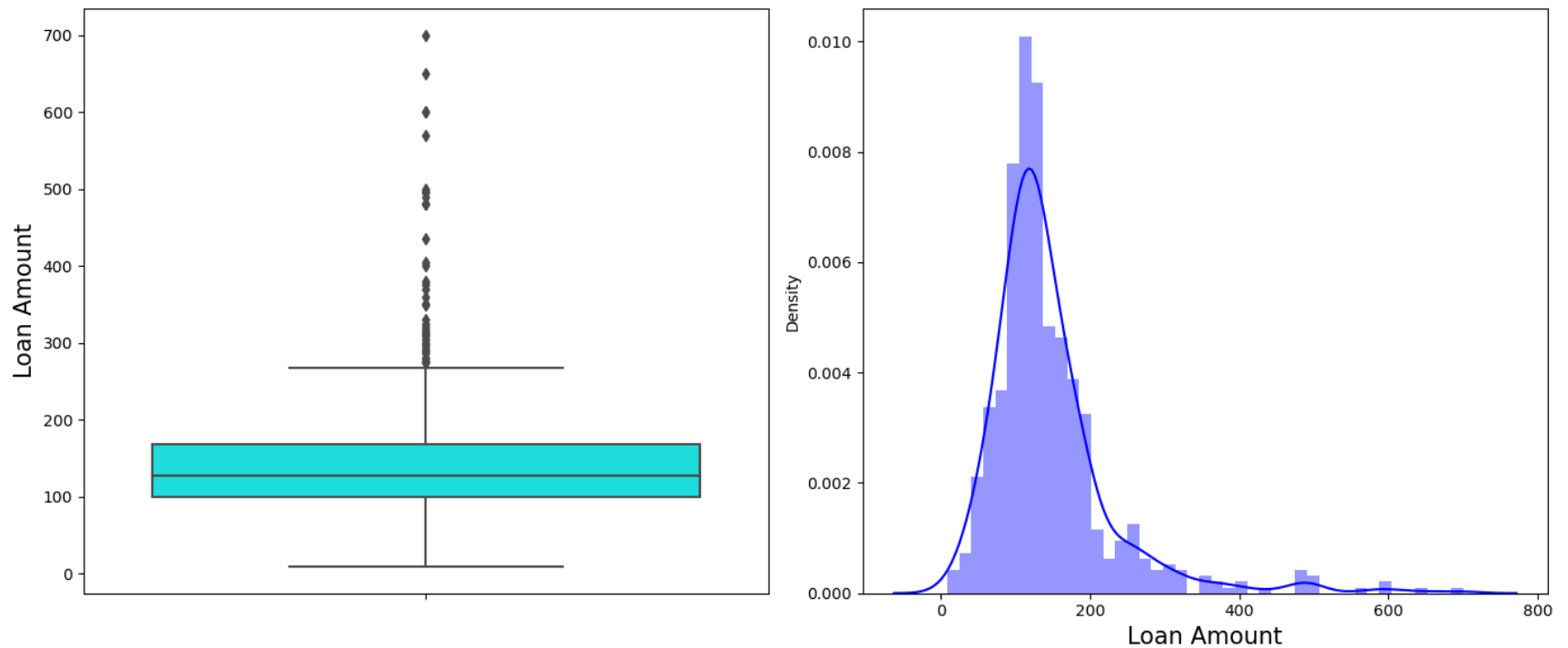
- Out of Total loan application 80 % applicants are Male. **We can Explore loan amount for each gender applied and evaluate whether on the same basis loan is approved for each gender or not?**
- Only 20% applicants are self employed. **So it will interesting to gain insight on relation between Applicant income and loan approval for non self employed category. We will look to find any benchmark range of Income for loan approval. Another benchmark we will try to find is about loan requirement for these two categories.**
- Nearly 70% are married and 75% of loan applicants are graduates
- Almost 60% of the applicants have no dependents.
- Most of applicants come from Semi Urban areas, followed by Urban and Rural areas.
- 80% people previously have credit history. Normally people having credit history are seen more prone to get loan approval.
- Nearly 70 % applicant gets loan approved.

We can impute categorical variable with mode in that category. For numerical variable we have option of mean and median. If Outliers are to stretch then we will impute with median.

Let check outliers for missing values Numerical variable having missing values by plotting boxplot.

```
In [10]: plt.figure(figsize=(14,6))
plt.subplot(1,2,1)
sns.boxplot(y='LoanAmount', data=df,color='cyan')
plt.ylabel('Loan Amount',fontsize=15)
plt.subplot(1,2,2)
sns.distplot(df['LoanAmount'], color='b')
plt.xlabel('Loan Amount',fontsize=15)
```

```
plt.tight_layout()
plt.show()
```



```
In [35]: print("Mean of Loan Amount:",df['LoanAmount'].mean())
print("Median of Loan Amount:",df['LoanAmount'].median())
```

Mean of Loan Amount: 146.41216216216216
Median of Loan Amount: 128.0

Comment -

- The mean is greater than median loan amount.
- Clearly we can see outliers in boxplot and feature is stretch to far in distribution plot.

As extreme outliers are present in feature and for that reason as data is more sensitive to mean we are going to impute missing values in loan amount with median.

Imputation of Missing values

Imputation details :

1. Missing values in Loan amount is impute with median value.
2. Maximum Loan term is 360 Months so Missing value in Loan amount term is replace with 360 Months.
3. Credit History, Self Employed, dependents, Gender and Married are replace with mode of repective features.

```
In [11]: # Imputating Missing value with mode for categorical features
df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0], inplace=True)
df['Dependents'].fillna(df['Dependents'].mode()[0], inplace=True)
df['Gender'].fillna('-1', inplace=True)
df['Married'].fillna(df['Married'].mode()[0], inplace=True)

# Imputation of Numerical features
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0], inplace=True)
df['LoanAmount'].fillna(df['LoanAmount'].median(), inplace=True)
```

Missing Value Check After Imputation

```
In [12]: # missing_values = df.isnull().sum().sort_values(ascending = False)
# percentage_missing_values =(missing_values/len(df))*100
# print(pd.concat([missing_values, percentage_missing_values], axis =1, keys =['Missing Values', '% Missing data']))

df.isnull().sum()
```

```
Out[12]: Gender                0
Married                0
Dependents             0
Education              0
Self_Employed          0
ApplicantIncome        0
CoapplicantIncome      0
LoanAmount             0
Loan_Amount_Term       0
Credit_History         0
Property_Area          0
Loan_Status            0
dtype: int64
```

Comment :

Finally, No Missing Value is Present.

We are Now Yes To Go Further !!!

Statistical Matrix

```
In [7]: df.describe()
```

```
Out[7]:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	614.000000	614.000000	614.000000
mean	5403.459283	1621.245798	145.752443	342.410423	0.855049
std	6109.041673	2926.248369	84.107233	64.428629	0.352339
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.250000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	164.750000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000

Comment:

- In Applicant Income & Coapplicant Income Std deviation value is greater than median. So data is spread and skewed.
- Taking 75% and Max rows into consideration we can surely say that Outliers exist in Applicant Income, Coapplicant Income, Loan Amount.
- Since Credit History is Categorical variable there is no significance in different statistical parameter of it.
- Minimum Tenure for Loan is 12 Months and Maximum Loan tenure is 480 Months.
- Minimum Applicant income is 150 and maximum is 81000.

Let dive into exploration of Target and independent feature.

Target Variable

Encoding categorical data

```
In [13]: df["Dependents"]=df["Dependents"].replace("3+",3)
df["Dependents"]=df["Dependents"].astype(int)
```

```
In [14]: # Using Label Encoder on categorical variable
from sklearn.preprocessing import LabelEncoder
Category=df.select_dtypes(include="object")
le = LabelEncoder()
for i in Category:
    df[i] = le.fit_transform(df[i])
df.head()
```

```
Out[14]:
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_H
0	1	0	0	0	0	5849	0.0	128.0	360.0	
1	1	1	1	0	0	4583	1508.0	128.0	360.0	
2	1	1	0	0	1	3000	0.0	66.0	360.0	
3	1	1	0	1	0	2583	2358.0	120.0	360.0	
4	1	0	0	0	0	6000	0.0	141.0	360.0	

```
In [17]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                614 non-null   int32
1   Married               614 non-null   int32
2   Dependents            614 non-null   int32
3   Education             614 non-null   int32
4   Self_Employed         614 non-null   int32
5   ApplicantIncome       614 non-null   int64
6   CoapplicantIncome     614 non-null   float64
7   LoanAmount            614 non-null   float64
8   Loan_Amount_Term      614 non-null   float64
9   Credit_History        614 non-null   float64
10  Property_Area         614 non-null   int32
11  Loan_Status           614 non-null   int32
dtypes: float64(4), int32(7), int64(1)
memory usage: 40.9 KB

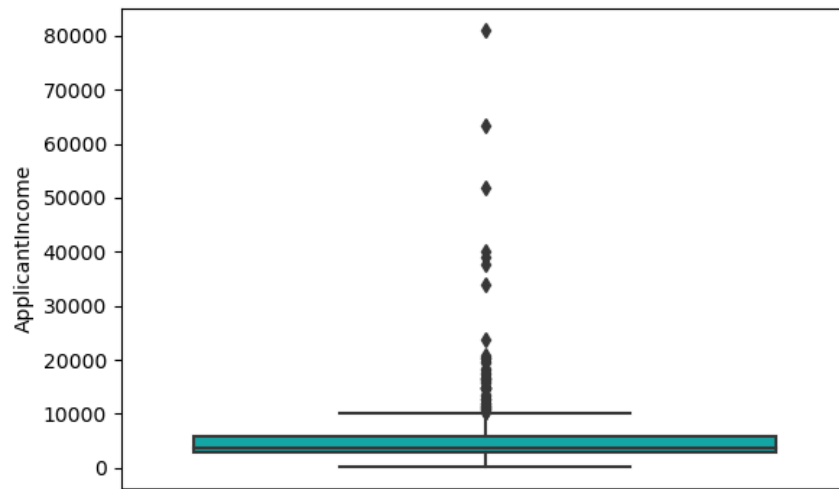
```

Feature selection and Engineering

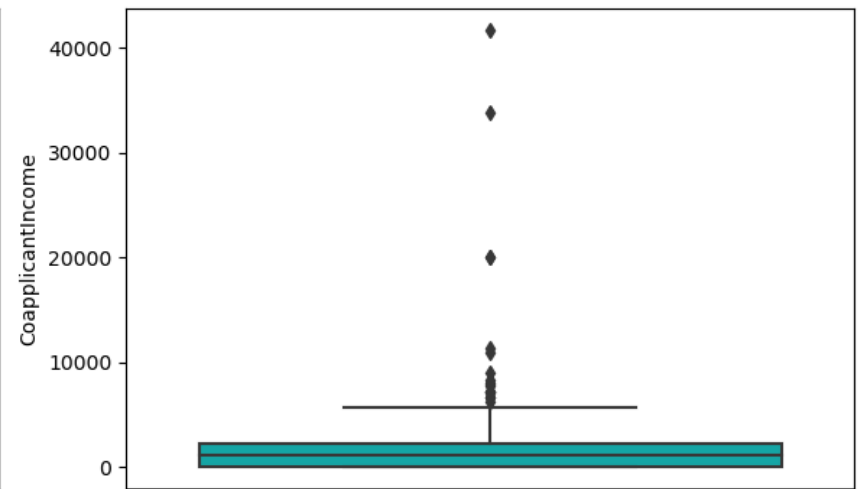
1. Outliers Detection and Removal

```
In [15]: Numerical = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term']
```

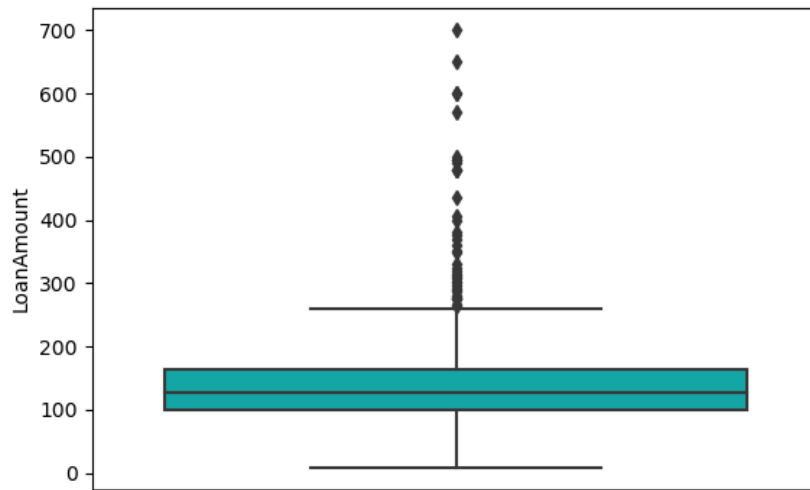
```
In [16]: plt.figure(figsize=(12,8),facecolor='white')
plotnumber=1
for column in Numerical:
    if plotnumber<=4:
        ax=plt.subplot(2,2,plotnumber)
        sns.boxplot(y=df[column],color='c')
        plt.xlabel(column,fontsize=20)
        plotnumber+=1
plt.tight_layout()
plt.show()
```



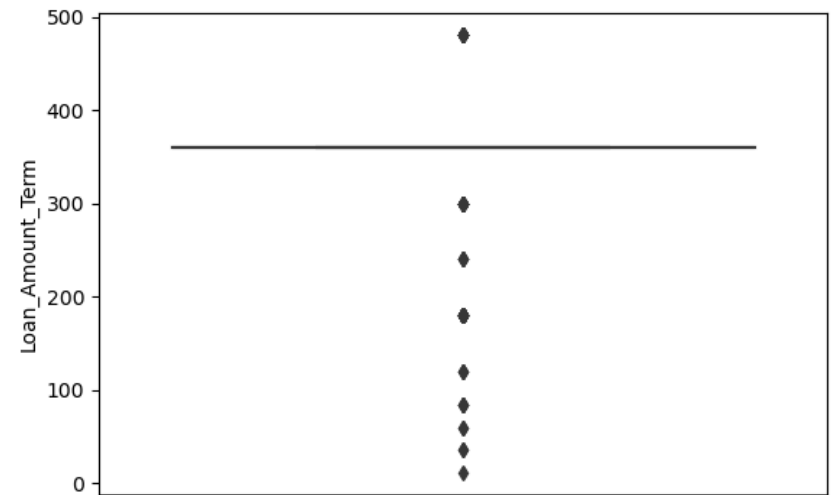
ApplicantIncome



CoapplicantIncome



LoanAmount



Loan_Amount_Term

From Boxplot we can see outliers exist dataset.

```
In [17]: # Dropping unnecessary columns
df.drop(["Loan_ID"], axis=1, inplace=True)
```

```
In [18]: from scipy.stats import zscore
z = np.abs(zscore(df))
df1 = df[(z<3).all(axis = 1)]
```

```
print ("Shape of the dataframe before removing outliers: ", df.shape)
print ("Shape of the dataframe after removing outliers: ", df1.shape)
print ("Percentage of data loss post outlier removal: ", (df.shape[0]-df1.shape[0])/df.shape[0]*100)

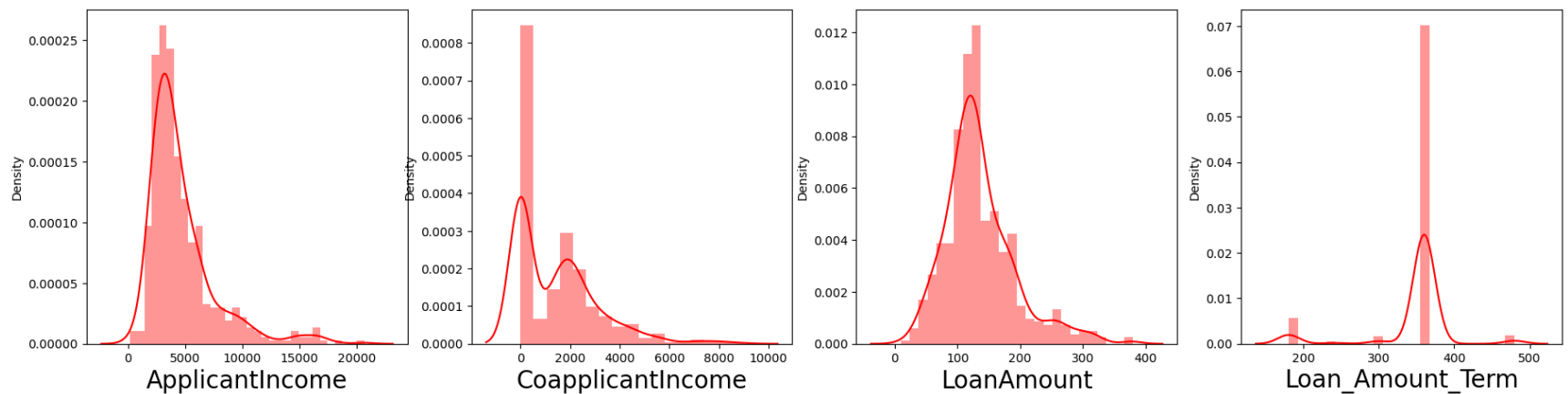
df=df1.copy() # reassigning the changed dataframe name to our original dataframe name
```

Shape of the dataframe before removing outliers: (614, 12)
 Shape of the dataframe after removing outliers: (577, 12)
 Percentage of data loss post outlier removal: 6.026058631921824

Data Loss

2. Skewness of features

```
In [19]: plt.figure(figsize=(22,5),facecolor='white')
plotnum=1
for col in Numerical:
    if plotnum<=4:
        plt.subplot(1,4,plotnum)
        sns.distplot(df[col],color='r')
        plt.xlabel(col,fontsize=20)
        plotnum+=1
plt.show()
```



```
In [32]: df.skew()
```



```
Out[32]: Gender          -1.622920
Married        -0.630211
Dependents      1.052106
Education       1.306588
Self_Employed   2.252848
ApplicantIncome 2.148522
CoapplicantIncome 1.350517
LoanAmount      1.151525
Loan_Amount_Term -2.098806
Credit_History -1.976043
Property_Area   -0.055332
Loan_Status     -0.822635
dtype: float64
```

Comment :

- Out of all above feature 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount' are skewed which are numerical feature.
- Other features are categorical in nature so skewness is nothing to do with these remaining feature. We will ignore them.
- We will yeo-johnson transformation method.

```
In [20]: # Removing skewness using yeo-johnson method to get better prediction
skew = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']

from sklearn.preprocessing import PowerTransformer
scaler = PowerTransformer(method='yeo-johnson')
```

```
In [21]: df[skew] = scaler.fit_transform(df[skew].values)
df[skew].head()
```

```
Out[21]:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount
0	0.681780	-1.122446	0.000771
1	0.234783	0.744117	0.000771
2	-0.527201	-1.122446	-1.437336
3	-0.791972	0.895786	-0.153545
4	0.728848	-1.122446	0.238260

Checking skewness after using yeo-johnson ethod

```
In [22]: df.skew()
```

```
Out[22]: Gender          -1.622920  
Married          -0.630211  
Dependents        1.052106  
Education         1.306588  
Self_Employed     2.252848  
ApplicantIncome   0.027981  
CoapplicantIncome -0.191876  
LoanAmount        0.048425  
Loan_Amount_Term  -2.098806  
Credit_History    -1.976043  
Property_Area     -0.055332  
Loan_Status       -0.822635  
dtype: float64
```

For Numerical variable skewness is within permissible limit.

**So Yes To Go Forward !!! **

3. Correlation

```
In [38]: df.corr()
```

Out[38]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_A
Gender	1.000000	0.378997	0.188162	0.045696	-0.006207	0.058590	0.234551	0.172146	
Married	0.378997	1.000000	0.329900	0.024817	-0.015779	-0.024783	0.335820	0.181878	
Dependents	0.188162	0.329900	1.000000	0.069814	0.044543	0.105994	0.004109	0.131772	
Education	0.045696	0.024817	0.069814	1.000000	-0.007139	-0.176074	0.049739	-0.128715	
Self_Employed	-0.006207	-0.015779	0.044543	-0.007139	1.000000	0.212260	-0.087338	0.117218	
ApplicantIncome	0.058590	-0.024783	0.105994	-0.176074	0.212260	1.000000	-0.360946	0.432154	
CoapplicantIncome	0.234551	0.335820	0.004109	0.049739	-0.087338	-0.360946	1.000000	0.200081	
LoanAmount	0.172146	0.181878	0.131772	-0.128715	0.117218	0.432154	0.200081	1.000000	
Loan_Amount_Term	-0.104983	-0.127348	-0.087389	-0.090523	-0.032914	-0.069429	0.000951	0.049057	
Credit_History	0.013172	0.019308	-0.020288	-0.075217	-0.016390	0.028825	0.006564	-0.003626	
Property_Area	-0.026340	0.010595	0.002327	-0.068596	-0.028253	-0.011364	-0.074476	-0.098090	
Loan_Status	0.017408	0.089026	0.017872	-0.092658	-0.026525	-0.002484	0.079344	-0.023609	

Observation:

Most of feature are poorly or moderately correlated with target variable expect Credit History.

- Maximum correlation of 0.561 exist between Credit History and Loan status.

4. Checking Multicollinearity between features using variance_inflation_factor

All features VIF is within permissible limit of 10.

So No Need to Worry About Multicollinearity.

5. Balanceing Imbalanced target feature

```
In [45]: df.Loan_Status.value_counts()
```

```
Out[45]: 1    398
         0    179
         Name: Loan_Status, dtype: int64
```

As Target variable data is Imbalanced in nature we will need to balance target variable.

Balancing using SMOTE

```
In [47]: # Splitting data in target and dependent feature
X = df.drop(['Loan_Status'], axis =1)
Y = df['Loan_Status']
```

We have successfully resolved the class imbalanced problem and now all the categories have same data ensuring that the ML model does not get biased towards one category.

Standard Scaling

```
In [50]: from sklearn.preprocessing import StandardScaler
scaler= StandardScaler()
X_scale = scaler.fit_transform(X)
```

Machine Learning Model Building

```
In [52]: from sklearn.model_selection import train_test_split
         from sklearn.linear_model import LogisticRegression
         from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, f1_score
```

```
In [53]: X_train, X_test, Y_train, Y_test = train_test_split(X_scale, Y, random_state=99, test_size=.3)
         print('Training feature matrix size:', X_train.shape)
         print('Training target vector size:', Y_train.shape)
         print('Test feature matrix size:', X_test.shape)
         print('Test target vector size:', Y_test.shape)
```

```
Training feature matrix size: (557, 11)
Training target vector size: (557,)
Test feature matrix size: (239, 11)
Test target vector size: (239,)
```

Finding best Random state

Logistics Regression Model

```
In [57]: X_train, X_test, Y_train, Y_test = train_test_split(X_scale, Y, random_state=78, test_size=.3)
log_reg=LogisticRegression()
log_reg.fit(X_train,Y_train)
y_pred=log_reg.predict(X_test)
```

Logistics Regression Evaluation

Accuracy Score of Logistics Regression : 0.8158995815899581

Confusion matrix of Logistics Regression :

```
[[ 76  34]
 [ 10 119]]
```

classification Report of Logistics Regression

	precision	recall	f1-score	support
0	0.88	0.69	0.78	110
1	0.78	0.92	0.84	129
accuracy			0.82	239
macro avg	0.83	0.81	0.81	239
weighted avg	0.83	0.82	0.81	239

```
In [ ]: #from sklearn.metrics import classification_report
print(classification_report(y_pred,y_test))
```